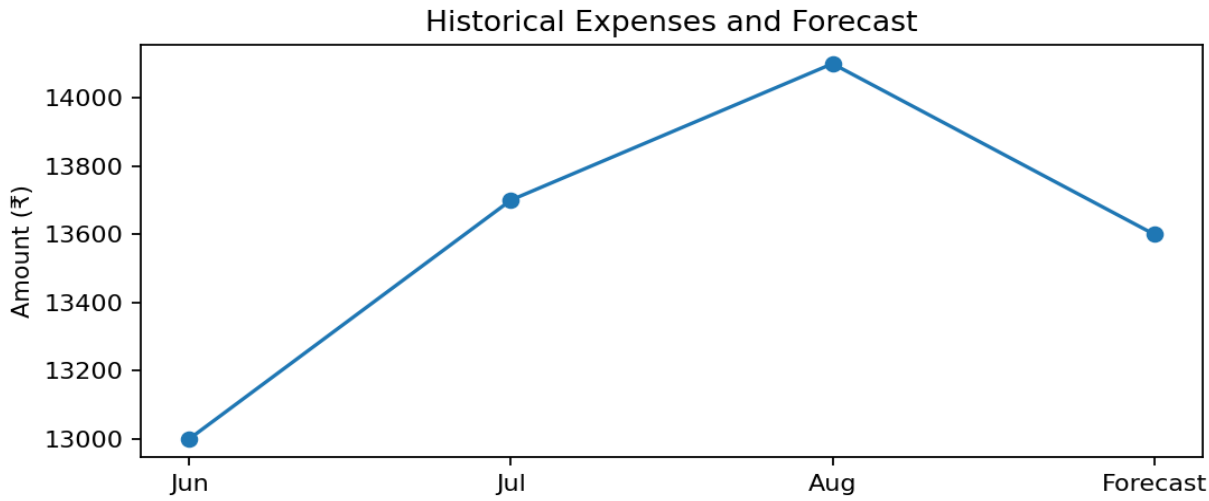
# 5.1 Expense Prediction

## Future Expense Forecasting

Future Expense Forecasting uses previous transaction information to estimate possible expenses in an upcoming period. The system can consider recurring payments, regular purchases, and recent spending averages. The forecast gives users an early idea of how much they may need to spend before the month ends.

### Objectives

- Analyze previous transaction records.
- Identify recurring or regular expenses.
- Calculate an estimated future expense.
- Present the forecast in a simple form.
- Support better preparation for upcoming payments.




### Working Procedure

1. Collect historical transaction data.
2. Group transactions by category or month.
3. Calculate recent averages and identify recurring payments.
4. Estimate the next period's expenses.
5. Display the forecast to the user.

### Example Data Table

| Month | Food | Transport | Bills | Total |
|---|---|---|---|---|
| June | ■6,000 | ■3,000 | ■4,000 | ■13,000 |
| July | ■6,500 | ■3,200 | ■4,000 | ■13,700 |

| Month | Food | Transport | Bills | Total |
|---|---|---|---|---|
| August | ■7,000 | ■3,100 | ■4,000 | ■14,100 |
| Estimated September | ■6,500 | ■3,100 | ■4,000 | ■13,600 |

## Python Implementation

```
previous_expenses = [13000, 13700, 14100]

average = sum(previous_expenses) / len(previous_expenses)

print("Previous expenses:", previous_expenses)
print("Estimated next-month expense:", round(average, 2))

if average > 14000:
print("Plan for higher expenses.")
else:
print("Expected expenses are relatively stable.")
```

## Benefits

Forecasting helps users prepare for upcoming expenses, especially recurring payments. It can also reduce surprises by showing an estimated expense amount before the next period begins.

Note: The values shown are illustrative examples for an academic project. A real application can use transaction records stored in a database.